Université Le Havre Normandie

Spécialité

Master 2 IMFL

U.F.R. des Sciences et Techniques

BEDDAR Rebiha

Conte rendu à rendre pour le 8 Avril 2018.

Sujet :

Optimisation de portefeuille avec diversification avec les deux algorithmes (recuit simulé et génétique).

Proposé par:

Mr.I.Diarrassouba.

Introduction :

Les risques financiers occupent une place importante dans les activités financières et économiques, la diversification est une technique qui permet de réduire le risque, l'investisseur doit diversifier son portefeuille par secteur, classe d'actif et par zone géographique pour optimiser son potentiel de gain, dans notre cas d'étude on va parler sur la diversification de portefeuille par classe d'actif alors :

Qu'est ce qu'une diversification de portefeuille par classe d'actif ?

En finance la diversification est le processus d'allocation du capital de manière à réduire l'exposition à un actif ou à un risque particulier c'est-à-dire investir dans différents types d'actif dans le but de réduire le risque ou la volatilité en investissant dans une variété d'actifs. Si les prix des actifs ne changent pas en parfaite synchronie, un portefeuille diversifié aura moins de variance que la variance moyenne pondérée de ses actifs constitutifs, et souvent moins de volatilité que la volatilité de ses composants.

Problème donné :

On considère un ensemble de n actifs financiers. Les actifs sont classés en 3 types : A, B et C. On note $w_1, ..., w_n$, les quantités de chaque actif qui vont composer le portefeuille. Le critère de Herfindahl sur le portefeuille dont la composition est donnée par $W = (w_1, ..., w_n)$, correspond à

$$H(W)=\sum_{i=1}^{n} w_i^2$$

H(W) l'indice d"Herfindahl-Hirschmann est compris entre 0 et 1 il est interprété de telle sorte que lorsqu'il tend vers 0, on dit que le portefeuille est plus diversifié.

Donc pour assurer la diversification de portefeuille on va essayer de minimiser cette quantité H(W). Les actifs sont modélisés comme des variables aléatoires d'espérances mathématiques μ_i et d'écart-type σ_i , pour tout $i \in \{1, ..., n\}$. On modélise le risque associé à ce portefeuille par son écart-type, celui-ci donné par

$$\sigma p= \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \sigma_{ij}}$$

où σ_{ij} est la covarian des actifs i et j.

Pour finir, les actifs étant de 3 types A, B, C, différents, les indices {1, ..., n} des actifs sont partitionnés en trois sous-ensembles A, B et C , où A est

l'ensemble des indices des actifs de type A, B l'ensemble des indices des actifs de type B, et C celui des actifs de type C.

Formulation mathématique :

Le problème est formulé grâce au programme mathématique suivant.

Minimiser : $H(W)=\sum_{i=0}^{n} w_i^2$.

sous constraintes :

$$\sum_{i=1}^{n} w_i = 1 \quad (1)$$

$$0 \leq \sum_{i \in A}^{n} w_i \leq \eta A \quad (2)$$

$$0 \leq \sum_{i \in B}^{n} w_i \leq \eta B \quad (3)$$

$$0 \leq \sum_{i \in C}^{n} w_i \leq \eta C \quad (4)$$

$$\sqrt{\sum_{i \in A}^{n} \sum_{j \in A}^{n} w_i w_j \sigma_{ij}} / \sigma p \leq \frac{SA}{100} \quad (5)$$

$$\sqrt{\sum_{i \in B}^{n} \sum_{j \in B}^{n} w_i w_j \sigma_{ij}} / \sigma p \leq \frac{SB}{100} \quad (6)$$

$$\sqrt{\sum_{i \in C}^{n} \sum_{j \in C}^{n} w_i w_j \sigma_{ij}} / \sigma p \leq \frac{SC}{100} \quad (7)$$

$$-\varepsilon \leq W_i \leq \varepsilon \quad , \quad i \in \{1, ..., n\} \quad (8)$$

La contrainte (1) indique que la totalité du budget doit être utilisé pour composer le portefeuille et que les w_i donne la proportion du budget total alloué à l'actif i. Les contraintes (2), (3) et (4) donnent le budget maximum alloué à chaque type d'actif. Les contraintes (5), (6) et (7) indiquent les seuils risques à ne pas dépasser sur chaque type d'actif. SA , SB et SC donnent, en pourcentage, le seuil de risque pour chaque type d'actif. Les contraintes (8)donnent les bornes sur les variables w_i . Ici, ε est une constante fixée.

Pour résoudre numériquement ce problème d'optimisation avec contraintes on va appliquer deux algorithmes, l'algorithme de recuit simulé et l'algorithme génétique ensuite on va comparer les résultats obtenus

Le recuit simulé: permet de résoudre les problèmes d'optimisations c'est un algorithme inspiré des cristaux, son principe est d'explore l'espace des solutions en partant d'un point et en se "déplaçant" de proche en proche, le recuit simulé est donc basé sur la notion de voisinage d'une solution: les voisins d'une solution seront

des solutions proches de celle-ci, i.e. qu'on peut obtenir par exemple en transformant à peine la solution initiale.

Le recuit simulé consiste à:

1. Attribuer une température (virtuelle) au système (=le chauffer),
2. Choisir la solution initiale au hasard, et mesurer le score obtenu,
3. Modifier légèrement les paramètres (c'est-à-dire explorer les voisin) et mesurer à nouveau le score obtenu , dans notre cas pour choisir un voisin proche à un point j'ai tester deux méthodes.
   la première méthode :récupérer le même point(vecteur) retrancher une moitié d'une valeur d'un actif d'un type choisis aléatoirement et cette moitié est divisée encore une fois par deux une moitié est rajouté pour un actif de type différent de celui qui précède cet actif est choisi aléatoirement aussi et l'autre moitié est rajouté pour un actif choisi aléatoirement de dernier type qui reste et pour les autres valeurs sont gardés. **la deuxième méthode** consiste à choisir un autre voisin aléatoirement sans utiliser le point précédent comme toute les valeurs de vecteur W je les ai choisis dans l'intervalle]0 1[donc automatiquement ce voisin va être proche de point qui le précède.
4. Si le cout est meilleur (plus bas), on conserve à nouveau ces nouveaux paramètres, si le cout est moins bon (plus élevé), on conserve ces nouveaux paramètres avec une nouvelle probabilité proportionnelle à la température,
5. Refroidir un petit peux le système,
6. Recommencer à l'étape 2.

Algorithme génétique:

Les algorithmes génétiques, créés par J. Holland (75) puis développés par David Goldberg, sont basés sur deux principes de génétique: la survie des individus les mieux adaptés et la recombinaison génétique.
Le principe de base est de simuler l'évolution d'une population de solutions avec les règles citées ci-dessus en vue d'obtenir une solution ou un ensemble de solutions les plus adaptées. "A chaque génération, un nouvel ensemble de créatures artificielles est créé en utilisant des parties des meilleures solutions précédentes avec éventuellement des parties innovatrices." (Goldberg)

1. La fonction objectif s'appelle fonction d'adaptation.
2. La prochaine génération est est produite par: 1-sélection, 2- croisement et 3-mutation.

Algorithme_Génétique(k,nbr_iter) /*L'objectif: minimiser la fonction H(W)*/

1. Population = matrice de taille(k*n) représente ensemble de k vecteurs de solutions (les parents) générés aléatoirement
2. Tant que on a pas atteint le nombre d'itération encore:

3. W= un nœud de taille $n=n_1 + n_2 + n_3$ pris dans la population selon une probabilité qui diminue selon la fonction objectif H(W).

   $Pi=H(W)/\sum_{Y\in population} H(Y)$

   Dans notre cas j'ai pas divisé par $\sum_{Y\in population} H(Y)$ car cette somme est trop proche de 0 donc ça donne pas de résultat.

4. W'= un nœud différent de W de taille n pris dans la population de la même façon.
5. W"= résultat de croisement entre W et W', le croisement est fait aléatoirement : en effet en parcourant toute la population j'ai choisis aléatoirement le type d'actif soit type 1(A), type 2(B) ou type3(C) et j'ai fait une permutation entre tout les actifs de type choisi de W et W'.
6. Ajouter W" à la nouvelle population.
7. Population = nouvelle population.
8. Retourner la solution avec la valeur de H(W) la plus basse.

Analyse des résultats de la simulation numérique:

- Algorithme de recuit simulé :

Pour une température initiale =10
Résultat obtenu d'algorithme de recuit simulé pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) par la deuxième méthode de choix de voisinage :

```
*****************
Affichage de la Valeur initiale: 0.0818889
le Type 1 0.0758967
le Type 1 0.0931647
le Type 1 0.0927462
le Type 1 0.108505
le Type 1 0.00943731
le Type 2 0.099706
le Type 2 0.0435446
le Type 2 0.0676745
le Type 2 0.0908763
le Type 2 0.0280081
le Type 3 0.0347621
le Type 3 0.104853
le Type 3 0.0490294
le Type 3 0.0803656
le Type 3 0.0214297
*****************
*****Affichage de la solution finale*******
Valeur solution: 0.0815461
le Type 1 0.0758967
le Type 1 0.0931647
le Type 1 0.0927462
le Type 1 0.110405
le Type 1 0.00943731
le Type 2 0.099706
le Type 2 0.0435446
le Type 2 0.0676745
le Type 2 0.0828037
le Type 2 0.0280081
le Type 3 0.0347621
le Type 3 0.104853
le Type 3 0.0552025
le Type 3 0.0803656
le Type 3 0.0214297
*****La solutuion finale est realisable: false******
```

Résultat obtenu d'algorithme de recuit simulé pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) par la deuxième méthode de choix de voisinage :

```
*****************
Affichage de la Valeur initiale: 0.0985889
le Type 1 0.0387493
le Type 1 0.0221824
le Type 1 0.111542
le Type 1 0.147707
le Type 1 0.0729557
le Type 2 0.0311002
le Type 2 0.150454
le Type 2 0.00858659
le Type 2 0.0336001
le Type 2 0.0343091
le Type 3 0.0499198
le Type 3 0.0554561
le Type 3 0.0998489
le Type 3 0.122708
le Type 3 0.0208811
*****************
*****Affichage de la solution finale*******
Valeur solution: 0.0834128
le Type 1 0.0739279
le Type 1 0.0542614
le Type 1 0.0664995
le Type 1 0.00366074
le Type 1 0.0557391
le Type 2 0.109481
le Type 2 0.111138
le Type 2 0.0302486
le Type 2 0.106332
le Type 2 0.0679218
le Type 3 0.0889869
le Type 3 0.115519
le Type 3 0.0204787
le Type 3 0.0450659
le Type 3 0.0507393
*****La solutuion finale est realisable: false******
```

Avec la première méthode de choisir un voisin j'ai remarqué qu'il ya plus d'amélioration et elle explore plus de voisins que d'utiliser la deuxième méthode.

La gestion de la température :

Pour faire baisser la température à chaque itération je multiplié la température T fois mu=0.99.

Avec la première méthode de choisir un voisin j'ai fait varier la température initiale et j'ai remarqué plus que la température est plus grande j'ai obtenu plus de solutions courante pour améliorer la solution de ce problème d'optimisation, par exemple pour T=5 j'ai eu une seule solution qui est la solution initiale (0.881161) y'a pas de solution courante

Pour T=12 j'ai eu au moins une solution courante qu'est la solution finale(0.0769554) avec une solution initiale (0.0770008)

- Algorithme génétique: j'ai remarqué plus que j'augmente la taille de la population et le nombre d'itération je m'approche plus de la solution car logiquement si j'ai une population de taille grande j'aurai plus de chance

pour vérifier la meilleure solution parmi toutes les solutions qui peuvent être proposées.

Résultat obtenu d'algorithme génétique pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) pour 5 itérations et la taille de la population k=2:

```
Valeur de la solution finale: 0.136367
le Type 1 0.125667
le Type 1 0.0574517
le Type 1 0.12549
le Type 1 0.0598317
le Type 1 0.0463118
le Type 2 0.0615235
le Type 2 0.055501
le Type 2 0.167732
le Type 2 0.142611
le Type 2 0.161596
le Type 3 0.039841
le Type 3 0.0201023
le Type 3 0.0892146
le Type 3 0.0562741
le Type 3 0.0351264
```

Résultat obtenu d'algorithme génétique pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) pour 5 itérations et la taille de la population k=3:

```
Valeur de la solution finale: 0.12628
le Type 1 3.29551e-007
le Type 1 1.02811e-008
le Type 1 3.39504e-007
le Type 1 4.35817e-007
le Type 1 3.10881e-007
le Type 2 0.100439
le Type 2 0.0305258
le Type 2 0.0269726
le Type 2 0.0963609
le Type 2 0.152125
le Type 3 0.0448977
le Type 3 0.150661
le Type 3 0.153838
le Type 3 0.0778577
le Type 3 0.166322
*****La solutuion finale est realisable: false******
```

Résultat obtenu d'algorithme génétique pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) pour 10 itérations et la taille de la population k=3:

```
Valeur de la solution finale: 0.107116
le Type 1 0.143503
le Type 1 0.0448943
le Type 1 0.00519694
le Type 1 0.0703332
le Type 1 0.0766462
le Type 2 0.057894
le Type 2 0.150333
le Type 2 0.0570286
le Type 2 0.115552
le Type 2 0.0150352
le Type 3 0.00606227
le Type 3 0.0122671
le Type 3 0.0931515
le Type 3 0.00378585
le Type 3 0.148317
```

Résultat obtenu d'algorithme génétique pour (5 actifs de type A, 5 actifs de Type B et 5 actifs de C) pour 15 itérations et la taille de la population k=3:

```
Valeur de la solution finale: 0.0515354
le Type 1 1.76697e-007
le Type 1 1.53254e-007
le Type 1 3.74461e-007
le Type 1 3.69159e-007
le Type 1 4.052e-007
le Type 2 0.00936507
le Type 2 0.0852011
le Type 2 0.0632668
le Type 2 0.099991
le Type 2 0.0877032
le Type 3 0.0523461
le Type 3 0.0654504
le Type 3 0.0684659
le Type 3 0.0848053
le Type 3 0.0599359
```

Comparaison les résultats obtenus de l'algorithme de recuit simulé et l'algorithme génétique pour différent nombre d'actifs :

Pour l'algorithme de recuit simulé j'ai travaillé toujours avec la première méthode de choisir un voisin.

Les valeurs des paramètres choisies sont :

La température T=16, nombre d'itérations pour l'algorithme génétique nbr_iter=16 et la taille de la population k=3

n_1 :nombre d'actifs de type A, n_2:nombre d'actifs de type B et n_3 :nombre d'actifs de type C

Nombre d'actifs	n_1=5 n_2=5 n_3=5	n_1=5 n_2=4 n_3=3	n_1=4 n_2=4 n_3=3	n_1= 1 n_2=1 n_3=1
Sol_recuit_simulé	0.0807937	0.118909	0.121112	0.333625
Sol_génetique	0.0879269	0.102453	0.256119	0.68706

D'après les résultats dans le tableau ci-dessus obtenues soit par l'algorithme de recuit simulé soit par l'algorithme génétique on remarque que plus qu'on augmente le nombre d'actifs plus que la valeur de la fonction H(W) diminue ce qui montre l'hypothèse de départ la diversification de portefeuille réduit le risque.

En même temps on remarque que l'algorithme de recuit simulé donne des résultats mieux que l'algorithme génétique des fois c'est le contraire.

Conclusion :

Le recuit simulé est stochastique: il ne produit pas toujours les mêmes résultats à différents exécutions car il est basé sur des tirages au sort.

Avantage :

- possibilité d'échapper d'un minimum local.
- agitation ajustable avec la température.

L'algorithme génétique utilisé pour obtenir une solution approchée à un problème d'optimisation, lorsqu'il n'existe pas de méthode exacte (ou que la solution est inconnue) pour le résoudre en un temps raisonnable

Limite:

- Le temps de calcul : par rapport à d'autres métaheuristiques, ils nécessitent de nombreux calculs, en particulier au niveau de la fonction d'évaluation et la taille de la population par exemple dans notre cas ça prend beaucoup de temps pour donner un résultat pour k>3

La réalisabilité des solutions : les contraintes 5,6 et 7 qui sont pas vérifiées c'est pour cela la méthode Realisable retourne false mais par contre les autres contrainte sont vérifiées.